

Un diagramma dei casi d'uso (Use Case Diagram) è uno strumento UML che descrive le interazioni tra gli attori esterni e le funzionalità offerte dal sistema. Il sistema modellato è quello di gestione di un hotel, con tre funzionalità principali: inserimento di un ospite, inserimento di una prenotazione e visualizzazione degli arrivi.

Il concetto centrale da fissare è quello di **attore**. In UML, un attore non è necessariamente una persona fisica: è qualsiasi entità esterna che interagisce con il sistema, che si tratti di un essere umano (come il receptionist) o di un altro sistema informatico. L'attore identificato è l'**User**, che rappresenta appunto il receptionist dell'hotel.

Un aspetto rilevante emerso è il **problema delle dimensioni**: quando le entità da gestire sono numerose (come le stanze di un hotel), il sistema rischia di diventare complesso da modellare e da implementare. La soluzione è quella di raggruppare i casi d'uso per attore, ovvero organizzare le funzionalità del sistema attorno a chi le utilizza, non attorno ai dati che manipolano. Questo approccio mantiene il diagramma leggibile e orientato al comportamento reale del sistema.

Il sistema è composto da più **componenti**, ognuno dei quali corrisponde a una **pagina** distinta dell'interfaccia. La notazione "N componenti → 1 pagina" suggerisce che più componenti logici possono essere aggregati in un'unica vista, un principio tipico delle architetture a componenti. Ogni caso d'uso del diagramma corrisponde quindi a un componente funzionale autonomo dell'applicazione.

Il punto di partenza del ragionamento è una domanda fondamentale che ogni sviluppatore dovrebbe porsi prima di scrivere una singola riga di codice: **"ho già un'API?"**. Questo approccio riflette una mentalità orientata al contratto: prima di implementare la logica, si verifica cosa il backend è già in grado di restituire. Nel caso specifico, la necessità da soddisfare è sapere **chi arriva oggi in hotel**. La soluzione è già parzialmente costruita: esiste un endpoint REST che risponde alla chiamata `GET /sbb/api/bookings/todaysarrivals/1`, il quale restituisce un array JSON con le prenotazioni attive per la data odierna.

La risposta dell'API rivela una struttura dati articolata su tre livelli. L'oggetto radice è il **BookingDTO**, che rappresenta la prenotazione nel suo complesso e contiene campi come `from`, `to`, `price`, `notes`, `id`, `guestId` e `roomId`. Al suo interno sono annidati altri due oggetti: il **GuestDTO**, che descrive l'ospite con i suoi dati anagrafici (nome, cognome, codice fiscale, data di nascita, indirizzo), e il **RoomDTO**, che descrive la stanza assegnata con il suo prezzo base, nome e descrizione.

Il componente **TodayArrivals** è il punto di ingresso della funzionalità "chi arriva oggi", e la sua logica di inizializzazione ruota attorno a due servizi iniettati: il **BookingService** e lo **UserService**.

Lo **UserService** ha un ruolo particolare: non si limita a fare chiamate HTTP, ma **mantiene uno stato reattivo** attraverso un **Signal**. Un Signal, nel contesto di Angular moderno, è un contenitore reattivo che notifica automaticamente i componenti quando il suo valore cambia. In questo caso specifico, il Signal è parametrizzato come `User | null`, il che significa che può contenere un oggetto utente oppure essere vuoto — situazione che si verifica quando nessun utente ha ancora effettuato il login.

Per questa fase di sviluppo e test, è stato **simulato il login** di Laura Leone, un'utente fittizia associata all'hotel numero 1. Questo approccio è comune durante lo sviluppo: si inietta un utente predefinito per poter testare il flusso senza costruire ancora l'intera infrastruttura di autenticazione.

computed vs. effect.

Entrambi sono **derivati da Signal**, nel senso che reagiscono automaticamente quando un Signal da cui dipendono cambia valore. La differenza sta nel loro scopo. **computed** serve a **calcolare e restituire un nuovo valore** a partire da uno o più Signal esistenti — si comporta come una funzione pura che produce un risultato. **effect**, invece, **esegue un'operazione con side effect** senza restituire nulla, ed è concettualmente assimilabile a un metodo **void**: viene eseguito per i suoi effetti collaterali, come fare una chiamata HTTP, aggiornare il DOM, o scrivere nel log.

Nel nostro **effect** è la scelta corretta perché l'obiettivo non è calcolare un valore derivato, ma **reagire al cambiamento dell'utente loggato eseguendo una chiamata al backend**. Ogni volta che il Signal `loggedUser` cambia — ad esempio quando il login viene completato — l'effect si riesegue, verifica che l'utente abbia un hotel associato, e avvia la sottoscrizione all'Observable restituito da `getTodayArrivals`. Il risultato della chiamata viene poi scritto nel Signal `bookings` tramite `.set()`, aggiornando così la vista in modo reattivo.

Questo pattern — un **effect** che legge un Signal e scatena una chiamata HTTP il cui risultato aggiorna un altro Signal — è uno dei modi più puliti per collegare lo strato reattivo di Angular con la comunicazione asincrona verso il backend.

Il template del componente `TodayArrivals` utilizza la direttiva `@for` di Angular per iterare sulla lista di prenotazioni contenuta nel Signal `bookings`. La sintassi `bookings()` con le parentesi tonde è fondamentale: in Angular, un Signal non espone il suo valore direttamente come proprietà, ma lo restituisce **invocandolo come funzione**. Il parametro `track booking.id` serve a ottimizzare il rendering — Angular usa l'identificatore univoco per capire quali elementi della lista sono cambiati, evitando di ricostruire l'intero DOM ad ogni aggiornamento.

Per ogni prenotazione viene istanziato il componente figlio `app-booking-row`, al quale viene passato l'oggetto `booking` tramite property binding (`[booking]="booking"`). Questo è il classico pattern di **comunicazione padre-figlio** in Angular: il componente padre gestisce i dati, il componente figlio si occupa esclusivamente della presentazione.

Il componente `BookingRow` è deliberatamente semplice — il commento nel codice lo definisce esplicitamente un **componente di sola visualizzazione**, paragonabile concettualmente a un metodo `viewBooking.render(booking)` in Java. Riceve un singolo dato dall'esterno e lo stampa, senza gestire stato proprio né logica di business. Per ricevere il dato dal padre usa `input.required<Booking>()`, che è la sintassi moderna di Angular basata su Signal per dichiarare un input obbligatorio: se il componente viene usato senza passargli una prenotazione, Angular genera un errore a compile time. Il commento paragona questo meccanismo al passaggio di un parametro Antani — una variabile ricevuta dall'esterno che il componente usa senza sapere né voler sapere da dove viene.